

Programación básica

Ejercicio 1: Añadir a una pagina HTML un script que muestre un `alert` de la siguiente forma:

- a) Añadimos el `alert` de forma interna sin función.
- b) Añadimos el `alert` en un evento en un botón.
- c) Igual que el punto anterior pero usamos una función.
- d) Igual pero la función en un archivo externo.

Ejercicio 2: Crear una página con una imagen y un botón. Añadir el código necesario para que cuando pulse el botón salga un `alert ("hola")` y cuando pase el ratón encima de la foto (`onmouseover`) salga un `alert ("adios")`.

Ejercicio 3: A través de un prompt, pide el nombre al usuario y saludalo con un alert de la siguiente forma: "*Bienvenid@ a mi página XXXXXX*" (siendo XXXXXX el nombre que ha introducido el usuario).

Ejercicio 4: Pedir dos valores numéricos con prompt y mostrar su suma, resta, multiplicación, división y resto.

Ejercicio 5: Crea un script que pida al usuario un numero entero positivo N mayor a 0. Hay que controlar que el numero introducido sea correcto (que sea entero, que sea positivo y que sea un número). Si no es así se volverá a pedir.

A continuación debe realizar lo siguiente:

- a) Calcular los divisores del numero N y mostrarlos.
- b) Calcular la suma de los cuadrados de los divisores obtenidos en el paso anterior y mostrarla.
- c) Indicar si esa suma es un cuadrado o no (con una frase por pantalla)

Nota: NO se pueden usar arrays en este ejercicio.

Ejercicio 6: Crea un script que pida al usuario una cadena y, a continuación, devuelva el carácter central de la palabra. Si la longitud de la palabra es impar, devuelve el carácter del medio. Si la longitud de la palabra es par, devuelve los 2 caracteres del medio.

Ejercicio 7: Crea una página web con un solo botón en el que, al pulsarlo, se le pida al usuario un año (pej: 1492). A continuación, el programa debe mostrar a través de una ventana modal a qué siglo pertenece el año introducido.

El siglo I abarca desde el año 1 hasta el año 100 inclusive, el siglo II desde el año 101 hasta el año 200 inclusive, etc.

Ejemplos:

1705 --> 18
1900 --> 19
1601 --> 17
2000 --> 20
2742 --> 28

Importante: intenta que la función que programes tenga el menor número de líneas posibles. MAXIMO 5 líneas contando llaves (y es mucho). Piensa bien.

Ejercicios de Arrays

Ejercicio 8: Crea una página web con un botón en el que, al pulsarlo, se genere un array con 10 números aleatorios entre 1 y 50.

A continuación el script pedirá un número entero positivo al usuario.

Finalmente, el script debe indicar si el número que ha introducido el usuario se encuentra en el array o no (Usa una frase coherente, no muestres solo `true` o `false`)

b) convierte el código anterior en una función llamada `enArray()` la cual recibirá como parámetro el array y el número a buscar y devolverá `true` o `false` en cada caso.

Ejercicio 9: Dado un array relleno valores de cualquier tipo. Crea una función que devuelva el mismo array con los valores en el orden contrario. **Importante:** no se puede usar `.reverse()`

Ej: Para el array `[10,5,"hola",-9.5]`, la salida será `[-9.5, "hola",5, 10]`

Ejercicio 10: Dado un array relleno con números positivos y negativos. Crea una función que devuelva la suma de todos los valores positivos del array.

Ej: Para el array `[2,-8,7,3,-9-3,5]`, la suma de los valores positivos es: $2+7+3+5=17$

Ejercicio 11: Escribe un programa que pida 10 números enteros y los almacene en un array. A continuación, debe averiguar:

- el valor máximo de ese array.
- el número de veces que aparece.
- Las posiciones en las que aparece.

Ejemplo: `[7,10,143,10,52,143,72,10,143,7]`

El valor máximo es el 143

Aparece 3 veces

Posiciones: 2,5,8

Ejercicio 12: Dado un array relleno de números enteros, crea una función que devuelva un nuevo array igual al primero pero eliminando el valor más pequeño del array. Además, hay que tener en cuenta que:

- No se modifica el array original, es decir, no se puede cambiar de orden los elementos que queden.
- Si hay varios elementos con el mismo valor, se elimina el primero que aparezca.
- Si el array queda vacío, se devuelve un array vacío.

Ejemplos:

- Para el array: [1,2,3,4,5], la salida debe ser: [2,3,4,5]
- Para el array: [5,3,2,1,4], la salida debe ser: [5,3,2,4]
- Para el array: [2,2,1,2,1], la salida debe ser: [2,2,2,1]

Ejercicio 13: Crea una página web con un solo botón en el que, al pulsarlo, le pida al usuario dos números enteros positivos. A continuación el script debe averiguar cual de los dos números es el mayor y cual es el menor.

Hecho eso, debe crear un array y rellenarlo con números que vayan desde el menor valor hasta el mayor valor (ambos incluidos).

Ejemplo: Para los valores 2 y 7, el array que se debe generar es: [2,3,4,5,6,7]

Ejercicio 14: Crea una función que reciba un array de valores enteros (positivos y negativos), y devuelva otro array con la suma parcial de cada elemento del array que se pasa como parámetro.

Nota: La suma parcial de un elemento del array será la suma de ese elemento y todos los anteriores a él.

Ej: Para el array [1,2,3,4,5,6], el array resultante será [1,3,6,10,15,21]

Ejercicio 15: Crea una función llamada `countBy` la cual recibirá dos números enteros positivos: X e Y. Esta función debe devolver lo siguiente:

- Si alguno de los números es negativo devolverá un array vacío.
- Si no, devolverá con array con los Y primeros múltiplos de X empezando por el 1 (incluido)

Ejercicio 16: Crea una función que reciba un texto de cualquier longitud y muestre el número total de las diferentes vocales que aparecen en dicho texto. Es decir, debe indicar número de 'a', número de 'e' ... número de 'u' y el número total de todas las vocales que aparece en el texto.

No tendrá en cuenta si la vocal está en mayúsculas o minúsculas.

Ejercicio 17: Escriba una función que reciba dos arrays y devuelva un nuevo array con los elementos que solo aparecen una vez en total (ya sea en el primer o en el segundo array). El orden debe ser: primero los que están en el primer array y luego los que están en el segundo.

Ejemplos:

```
[1, 2, 3, 3] y [3, 2, 1, 4, 5] ==> [4, 5]
["Ray", "Jose", "Dani"] y ["Dani", "Jose", "Ivan"]) ==> ["Ray", "Ivan"]
[77, "ciao"] y [78, 42, "ciao"]) ==> [77, 78, 42]
[1, 2, 3, 3] y [3, 2, 1, 4, 5, 4] ==> [4, 5]
```

Ejercicio 18: El ácido desoxirribonucleico, ADN, es la principal molécula de almacenamiento de información en los sistemas biológicos. Está compuesto por cuatro bases de ácido nucleico: guanina ("G"), citosina ("C"), adenina ("A") y timina ("T").

El ácido ribonucleico, ARN, es la principal molécula mensajera de las células. El ARN difiere ligeramente del ADN en su estructura química y no contiene timina. En el ARN, la timina se sustituye por otro ácido nucleico, el uracilo ("U").

Cree una función que traduzca una cadena dada de ADN a ARN.

Por ejemplo:

Si se introduce la cadena GCAT, a salida debe ser GCAU

Si se introduce la cadena GCATCGTA, a salida debe ser GCAUCGUA

A tener en cuenta: La cadena de entrada puede tener una longitud arbitraria, incluso puede estar vacía. Se garantiza que toda cadena de entrada es válida, es decir, que cada cadena de entrada sólo estará formada por 'G', 'C', 'A' y/o 'T' en cualquier orden.

Nota: Lo primero que debes preguntarte es ¿entiendes lo que se está pidiendo?

Ejercicio 19: Crea un script que filtre una lista de nombres y devuelva otra lista solo con los que son amigos tuyos.

Como eres una persona muy “especial”, tú solo eres amigo/a de aquellas personas cuyo nombre se componga exactamente de 4 letras.

Ejemplo:

```
Entrada=["Manu", "Alberto", "Marcos", "Javi", "Ricardo", "Gabi"]
Salida = ["Manu", "Javi", "Gabi"]
Entrada = ["María", "Laura", "Lucía", "Susana"]
Salida = []
```

Suponemos que los array son correctos y tienen nombres.
Importante: hay que respetar el orden de los nombres en la salida.

Ejercicio 20: Dado un array de números enteros (cualquier longitud del array), devuelva la suma mas grande entre dos números adyacentes.

Ejemplo:

```
let numeros=[2,4,1,5,6,3]
```

Resultado: 11 *//viene de sumar 5 y 6*

```
//Por si no sabes lo que hay que hacer:  
//Las sumas de cada par de números adyacentes son:  
//2+4 = 6  
//4+1 = 5  
//1+5 = 6  
//5+6 = 11  
//6+3 = 9
```

Ejercicio 21: Crea una función llamada `likes` la cual va a recibir como único parámetro un array de nombres. Esta función debe devolver una cadena siguiendo el patrón que se muestra en el ejemplo siguiente:

```
[ ] --> "no one likes this"  
["Peter"] --> "Peter likes this"  
["Jacob", "Alex"] --> "Jacob and Alex like this"  
["Max", "John", "Mark"] --> "Max, John and Mark like this"  
["Alex", "Jacob", "Mark", "Max"] --> "Alex, Jacob and 2 others like this"
```

Nota: para este ejercicio no se permiten usar funciones de arrays u operadores especiales de Javascript.

Ejercicio 22: Realiza un script que pida 10 números por teclado y que los almacene en un array. A continuación muestra el contenido de ese array junto al índice de cada numero (mira el dibujo). Usa la consola para ello.

Seguidamente el script pedirá dos posiciones a las que llamaremos *inicial* y *final*. Debes comprobar que *inicial* es menor que *final* y que ambos números están entre 0 y 9. Si no es así, vuelve a pedirlos.

A continuación, coloca el número de la posición inicial en la posición final, rotando el resto de números para que no se pierda ninguno.

Al final se debe mostrar el array resultante.

Por ejemplo, para *inicial* = 3 y *final* = 7:

Índice	0	1	2	3	4	5	6	7	8	9
Valor	20	5	7	4	32	9	2	14	11	6

Array inicial

Índice	0	1	2	3	4	5	6	7	8	9
Valor	6	20	5	7	32	9	2	4	14	11

Array final

Ejercicios de Arrays bidimensionales

Ejercicio 23: Crea una función a la que se le pasa una matriz de números (nos da lo mismo el tamaño) y un número.

- La función debe devolver si el numero indicado está en la matriz o no.
- La función debe devolver si el numero indicado está en la matriz y la posición en la que está. Si el número no está, devolverá la posición -1, -1

Ejercicio 24: Crea un programa que genere una matriz 3X4 de números enteros aleatorios entre 10 y 40 (ambos incluidos). A continuación debe mostrar por consola:

- Suma total de todos los elementos de la matriz.
- Media de todos los elementos de la matriz.
- El mayor y el menor de todos los elementos.
- La cantidad de números pares e impares que tiene.
- La suma de cada fila de la matriz.

Ejercicio 25: Una clase tiene 4 alumnos y cada alumno ha realizado 5 exámenes. Las notas se almacenan en una matriz donde:

- cada fila representa a un alumno
- cada columna representa un examen

Puedes crear la matriz “a mano” en el código. Llama a es matriz: “notas”.

Crea un programa que calcule:

- La media de cada alumno.
- Cuántos alumnos tienen la media igual o superior a 5
- Que alumno tiene la media más alta.

Ejemplo:

Para la matriz:

```
let notas = [  
    [7, 8, 6, 9],  
    [5, 4, 6, 5],  
    [9, 8, 10, 9],  
    [6, 7, 5, 8],  
    [3, 4, 2, 5]  
];
```

La salida debe ser:

```
Media alumno 1: 7.5  
Media alumno 2: 5  
Media alumno 3: 9  
Media alumno 4: 6.5  
Media alumno 5: 3.5
```

```
Alumnos aprobados: 4
```

```
Alumno con mayor media: alumno 3
```

Ejercicio 26: Crea un programa que pida el numero de filas y el numero de columnas al usuario. Ambos números deben ser positivos y estar entre 3 y 7. Si un numero no cumple esas condiciones, debe volver a pedirse (solo ese número).

A continuación:

- Crea una matriz llamada ORIGINAL con tantas filas y columnas como ha indicado el usuario.
- Rellena la matriz con números aleatorios entre 1 y 9 (ambos incluidos).
- Crea la matriz traspuesta de la matriz ORIGINAL. La matriz traspuesta es aquella que intercambia las filas por las columnas.

Ejemplo:

Para la matriz ORIGINAL:

```
1  2  3  4  
5  6  7  8  
9  9  8  8
```

La matriz traspuesta es:

```
1  5  9  
2  6  9  
3  7  8  
4  8  8
```

Ejercicios de Interesantes

EE1: Las mesas del restaurante

Un restaurante nos ha encargado una aplicación para colocar a los clientes en sus mesas. En una mesa se pueden sentar de 0 (mesa vacía) a 4 comensales (mesa llena).

El funcionamiento es el siguiente:

Cuando llega un cliente se le pregunta cuántos son. Como el programa no está preparado para colocar a grupos mayores a 4, si un cliente solicita una mesa con mas comensales (pej, 6), el programa dará el mensaje “Lo siento, no admitimos grupos de 6, haga grupos de 4 personas como máximo e intente de nuevo” y volverá a preguntar.

Para cada grupo nuevo que llega, se busca siempre la primera mesa libre (con 0 personas). Si no quedan mesas libres, se busca una donde haya hueco para todo el grupo (por ejemplo si el grupo es de dos personas, se podrá colocar en mesas donde haya una o dos personas).

Cada vez que se sientan nuevos clientes se debe mostrar el estado de las mesas. Los grupos no se pueden romper aunque haya huecos sueltos suficientes.

A tener en cuenta:

- El programa comienza pidiendo el numero de mesas que tiene el restaurante.
- Inicialmente, las mesas se cargan con valores aleatorios entre 0 y 4 y mostrará por pantalla como quedan las mesas inicialmente.
- El programa seguirá pidiendo comensales hasta que se introduzca un valor negativo.

Ejemplo de ejecución:

```
//El usuario ha metido un valor de 10

Estado de las mesas: 3 2 0 2 4 1 0 2 1 1

El usuario pide una mesa para 2.
Por favor, Siéntese en la mesa 3

Estado de las mesas: 3 2 2 2 4 1 0 2 1 1

El usuario pide una mesa para 4
Por favor, Siéntese en la mesa 7

Estado de las mesas: 3 2 2 2 4 1 4 2 1 1
```

El usuario pide una mesa para 3
Por favor, Siéntese en la mesa 6

Estado de las mesas: 3 2 2 2 4 **4** 4 2 1 1

El usuario pide una mesa para 4
Lo siento, no queda sitio en el restaurante.

Estado de las mesas: 3 2 2 2 4 1 **4** 2 1 1

EI2: Autómata Celular unidimensional

Supongamos una lista de valores binarios que representan la existencia de células vivas (1) o no (0). Esta lista evoluciona a lo largo del tiempo, creándose nuevas células vivas y/o muriendo otras.

Una celda de la lista evolucionará a partir del estado de sus celdas vecinas (la de su izquierda y la de su derecha) y de ella misma en el instante anterior.

Para ello se siguen las siguientes reglas:

- 000 – 0
- 001 – 1
- 010 – 1
- 011 – 0
- 100 – 1
- 101 – 1
- 110 – 0
- 111 – 0

Una regla $IXD - Y$ se interpreta de la siguiente forma: “La celda X rodeada de las celdas I y D , evolucionará al estado Y ”

Por ejemplo:

011 – 0 : una celda en el estado 1 rodeada de cero ala izquierda y 1 a la derecha, evolucionará a 0 en el siguiente paso.

101 – 1 : una celda en el estado 0 con rodeada de unos, evolucionará a 1 en el siguiente paso.

Vamos a crear un script que nos permita estudiar la evolución del autómatas celular antes descrito de tamaño TAM (numero entero mayor o igual a 3) Para ello debes crear las siguientes funciones:

- `inicializarAutomata` : llena el autómatas de células muertas.
- `mostrarEstado`: imprime el contenido del autómatas teniendo en cuenta que mostrará las células vivas con el carácter '*' y las muertas con '.'
- `primeraJugada`: indica a través de código qué celdas tienen inicialmente células vivas.

Haciendo uso de las funciones anteriores, crea un script que pide al usuario el tamaño del autómatas (valor entero mayor o igual a 3) y el número de pasos que va a dar el autómatas. Muestra por consola el estado del autómatas en cada paso que se vaya dando.

Ejemplo de salida:

Para un autómatas de tamaño 15, con la celda 8 como única viva en la primera jugada y 7 pasos, tenemos:

```
Paso 0: . . . . . * . . . . .
Paso 1: . . . . . * * * . . . . .
Paso 2: . . . . * . . . * . . . . .
Paso 3: . . . * * * . * * * . . . . .
Paso 4: . . * . . . * . . . * . . . .
Paso 5: . . * * * . * * * . * * * . .
Paso 6: . * . . . * . . . * . . . * .
```

El3 La Búsqueda del Tesoro

Crea un script que simule el siguiente juego:

En un tablero, NxM la maquina colocará de forma aleatoria varias minas y un tesoro. El usuario intentará averiguar la posición del tesoro indicando la posición del tablero que quiere mirar. Podrá seguir jugando mientras no encuentre una de las minas (muere) o el tesoro (gana).

Para este ejercicio usa un tablero de 4x5 con 3 minas.

Hay que asegurarse que tanto las minas como el tesoro no se colocan en posiciones ya ocupadas.

En cada iteración del juego se pintará el tablero y se preguntará al usuario qué coordenadas quiere mostrar del tablero.

Usa la consola para pintar el tablero de la siguiente forma:

- Si una coordenada aún no ha sido visitada, pinta un *
- Si una coordenada ha sido visitada y no tiene nada, pinta un _
- Si una coordenada ha sido visitada y tiene una mina, pinta una X (y mata al jugador)
- Si una coordenada ha sido visitada y tiene el tesoro, pinta un € (y el jugador gana)

Puedes pedir al usuario las coordenadas como quieras: ambos valores de golpe o de uno en uno. Lo único a tener en cuenta es que, para el usuario, las coordenadas empiezan en 1, no en 0.

Mejora: si hay una mina a una casilla de distancia de la posición que ha descubierto el usuario, muestra un aviso indicando que tenga cuidado (pero no reveles la posición de la mina).

Árbol DOM

Atributos de HTML

1.- Crea un enlace que vaya a google. Crea dos botones junto al enlace:

- Al pulsar el primero, muestra la URL a dónde va el enlace y su texto.
- Al pulsar el segundo, cambia la URL a otra diferente y su texto por “he cambiado”

2.- Crea una tabla en de 4 filas y 4 columnas. Usando atributos HTML, coloca un borde, unas medidas de 300x300px y una alineación al centro. A continuación crea tres botones:

- El primero muestra los valores de anchura y altura de la tabla, pide unos valores nuevos para cada propiedad y los aplica a la tabla. ¿se aplican correctamente los valores? ¿Falla alguno?
- El segundo muestra el valor actual del borde de la tabla y solicita un valor nuevo para aplicarlo. ¿Funciona correctamente el nuevo valor?
- El tercer botón va cambiar la alineación de la tabla cada vez que se pulse. Es decir, si la tabla está alineada al centro, al pulsar el botón, la tabla quedará alineada a la derecha. Si se pulsa el botón otra vez, se alineará a la izquierda y si vuelvo a pulsarlo, la tabla se pondrá en el centro.

3.- Crea un DIV con el número 0 en su interior. Bajo el DIV, coloca dos botones: uno para sumar y otro para restar.

- Si se pulsa el botón para sumar, se incrementa en uno el número que hay en el DIV en ese momento.
- Si se pulsa el botón restar, se decrementa en uno el número que hay en el DIV en ese momento.

4.- Crea una galería de fotos (al menos 5 fotos) que posea un botón para avanzar y otro para retroceder.

- El botón para avanzar hará que se muestre la siguiente foto cada vez que se pulse. Si se llega al final de la lista de imágenes, el botón deja de funcionar.
- El botón para retroceder hará que se muestre la foto anterior cada vez que se pulse. Si se llega al principio de la lista de imágenes, el botón deja de funcionar.

b) Mejorar el ejercicio haciendo que el botón correspondiente se deshabilite cuando se alcanza el final/principio de la lista de fotos.

5.- Crea un párrafo en HTML y ponle la clase “uno”. Usando CSS, coloca un fondo, un borde, una anchura y un color/tam de letra para esa clase. Crea la clase “dos” similar a la clase “uno” pero con valores diferentes.

- a) Cuando se haga click sobre el párrafo, cambia de la clase “uno” a la clase “dos”.
- b) Cada vez que haga click en el párrafo, cambia de clase alternativamente.

6.- Crea cinco botones y coloca como `value` un color diferente a cada uno. Cada vez que se pulse uno de los botones, el fondo de la página debe cambiar al color indicado por ese botón.

7.- Crea una tabla con 4 filas y 4 celdas por fila. Usando atributos de HTML, coloca un borde y un tamaño de 400x400px y deja el fondo en blanco.

- a) Crea el código Javascript necesario para que cada vez que se pulse sobre una celda, esa celda se vuelva de color negro. *Nota: hay que usar funciones anónimas y `this`.*
- b) Mejora el ejercicio anterior para que cada vez que se pulse sobre una celda, esta cambie a negro o a blanco alternativamente según corresponda.
- c) Super mejora: define una lista de 4 colores y ahora cada vez que se pulsa sobre una celda, esta va cambiando de color siguiendo la lista de colores definida (si llega al final de la lista, vuelve al principio).

8.- Implementa el efecto ROLLOVER sobre una imagen: al pasar el ratón por encima, cambia la imagen. Al sacar el cursor fuera, la imagen vuelve a su estado original.

Propiedades CSS

9- Repite el ejercicio 5 de esta relación pero esta vez no uses clases (toca directamente todas las propiedades CSS que necesites).

10.- Crea tres DIV de tamaño 100x100px y con un color de fondo diferente cada uno. Al pasar por encima de alguno de esos DIVs, el color de fondo de la página debe cambiar al color que tenga el DIV sobre el que está el cursor del ratón.

b) Añade el código necesario para que el color de la página vuelva a su color original si el cursor del ratón no está encima de ninguno de los DIVs del ejercicio.

11.- Crea tres DIV de tamaño 300x100px con un color de fondo distinto cada uno y con un texto en su interior que indique “Soy el DIV X” (siendo X un número del uno al tres).

- a) Oculta el segundo DIV usando la propiedad `visibility` de CSS. Consigue que ese DIV “vuelva a aparecer” sí y solo sí, el cursor del ratón se coloca en el primer DIV.
- b) Oculta el segundo DIV usando la propiedad `display` de CSS. Consigue que ese DIV “vuelva a aparecer” sí y solo sí, el cursor del ratón se coloca en el primer DIV.

12.- Coloca una imagen dentro de un DIV con `position: absolute`. Crea dos botones:

- Al pulsar uno de ellos la imagen debe moverse 5px hacia la derecha de la pantalla.
- Al pulsar el otro, la imagen debe moverse 5px hacia la izquierda de la pantalla.

Nota: Recuerda la posición (0,0) de la página web al usar top/left/right/bottom

b) Mejora el ejercicio haciendo que la imagen no pueda salirse de los límites de la pantalla. *Nota: realiza el cálculo a ojo, ya aprenderemos a obtener las medidas de la página correctamente.*

c) Realizar el mismo ejercicio pero esta vez la imagen se moverá hacia la derecha mientras el ratón esté encima de ella. Coloca un botón para resetear los valores de posición de la imagen.

13.- Crea el efecto SPOILER que se puede ver en foros y otras páginas.

Se oculta el contenido de un elemento (pej, un párrafo) y sólo se muestra el mensaje “Mostrar”. Al hacer click sobre dicho mensaje, se muestra el contenido del elemento y el mensaje cambia a “Ocultar”. Si se pulsa sobre ese mensaje, el elemento vuelve a su estado original (contenido oculto y mensaje “Mostrar”).

14.- Crear un menú vertical con efecto acordeón usando los siguientes apartados:

- Inicio
 - Portada.
 - Acerca de mi.
 - Mis fotos.
- Proyectos
 - Web Apps.
 - Mobile Apps.
 - Juegos.
- Portfolio
 - Páginas webs.
 - Software.
- Contacto

A tener en cuenta:

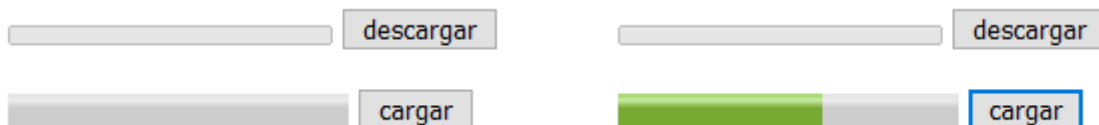
- El menú debe tener una maquetación en CSS.
- Los subapartados permanecen ocultos hasta que se pulsa sobre el apartado correspondiente.
- Al abrir un bloque de subapartados, se cierran los demás.

15.- Crear el mismo menú que el ejercicio anterior pero en horizontal. Ahora, al pasar el ratón por encima de cada apartado, aparece el bloque de subapartados correspondiente.

A tener en cuenta:

- El menú debe tener una maquetación en CSS.
- El bloque de subapartados quedará por encima del resto de elementos que tenga debajo (pej, sections, articles...), no desplazará al resto de elementos.
- Un bloque de subapartados desaparecerá cuando se pase por encima de otro apartado.

16.- Usando las funciones `setInterval` y `clearInterval` de Javascript. Coloca una etiqueta `meter`, una etiqueta `progress` y un botón al lado de cada una. Al pulsar el botón se debe simular la “carga” de la barra. Muestra un mensaje cuando se alcance el máximo.



b) Mejora el ejercicio haciendo que cuando pulsemos uno de los botones, este quede bloqueado hasta que la barra del elemento `progress/meter` se complete del todo.

17.- Carrera de DIVS

Coloca 4 divs de 100x100px y color de fondo distinto en la parte izquierda de la pantalla (uno debajo de otro). Crea un botón que se llame CORRER y al pulsarlo, los divs se moverán hacia la derecha de la pantalla de manera automática. Cada DIV debe moverse a una velocidad aleatoria que cambiará en cada carrera. El primer DIV que llegue a la derecha de la pantalla, gana. *Nota: realiza el cálculo a ojo, ya aprenderemos a obtener las medidas de la página correctamente.*

Coloca otro botón para resetear todo y volver a comenzar la carrera.

18.- Modifica el ejercicio 16 de esta relación de la siguiente forma:

- Usa sólo el elemento progress y el botón que se encarga de realizar el funcionamiento explicado en el ejercicio 16. Cambia el nombre del botón por “Generar moneda”
- Crea usando HTML un div en la parte superior izquierda llamado MONEDAS, cuyo contenido inicial será el número 0.
- Cada vez que se pulse sobre el botón ‘Generar moneda’, se bloqueará dicho botón y se cargará la barra de progress como se vio en el ejercicio 17.
- Al terminar la carga, se sumará una moneda al contenido del div MONEDAS y se desbloqueará el botón ‘Generar Moneda’.

Nodos DOM

19.- En HTML, crea una lista de 3 elementos y el botón “Añadir”. Al pulsar sobre el botón, se le pide al usuario que meta una cadena de texto y se creará una nueva entrada en la lista con el texto indicado.

20.- En HTML, crea un párrafo con varias líneas de texto, y tres botones: ‘Generar’, ‘Borrar Viejo’ , ‘Borrar Nuevos’ (deshabilitado) y ‘Sustituir’. Nada lleva ID así que no puedes usar *getElementById*.

En CSS, crea dos clases para maquetar el párrafo (pon anchura, altura, borde, tamaño de letra...) y haz que el párrafo anterior pertenezca a la primera clase.

Programa el siguiente funcionamiento de los botones:

- Generar: cada vez que se pulse, crea un nuevo párrafo similar al que ya existe pero maquetado con la segunda clase. Todos los párrafos generados serán hermanos.
- Borrar Viejo: elimina el párrafo que sale por defecto en la página. Al hacer eso, este botón y el botón ‘Sustituir’ quedan bloqueados.
- Borrar Nuevos: este botón está bloqueado al principio y se desbloquea al pulsar el botón ‘Generar’. Su cometido es borrar TODOS los párrafos generados por el botón generar.
- Sustituir: este botón sustituye el párrafo inicial por una tabla de dos filas y dos celdas con números en su interior. Recuerda que este botón estará bloqueado si se borra el párrafo inicial con el botón ‘Borrar Viejo’.

21.- Crea un programa en Javascript que al colocar el ratón encima de cualquier enlace, aparezca un div de 200x200px con maquetación en CSS (color de fondo, tamaño de letras, relleno, sombra paralela, bordes redondeados...) en la esquina superior derecha de la pantalla con el texto: *Esto es un enlace*.

b) Mejora el ejercicio anterior para que el mensaje sea: *“La URL de este enlace es: xxxxxx”* (siendo xxxxxx la url del enlace sobre el que se coloca el ratón).

22.- Crea una página web con un botón y un `<div>` vacío. Maquéalo de forma que tenga unas medidas de 100x100px, borde y el texto aparezca centrado tanto en horizontal como vertical.

Cada vez que el usuario pulse el botón, debe añadirse dentro del `<div>` un nuevo párrafo con el texto "Párrafo número X", donde X es un contador que empieza en 1 y aumenta con cada clic.

Tanto el nuevo nodo como su contenido, debe crearse dinámicamente usando las funciones adecuadas.

Cuando se recargue la página, el contador vuelve a empezar desde 1.

23.- Crea una página con un `<div>` que contenga una imagen y un párrafo descriptivo de la propia imagen.

Debajo, coloca un botón con el texto “Cambiar contenido”. Cuando el usuario pulse el botón, ambos elementos (imagen y texto) deben sustituirse por otros nuevos creados desde JavaScript (por ejemplo, una imagen distinta y un nuevo texto).

24.- Crea una lista (`<ul>`) con varios 7 elementos (`<li>`) representando tareas pendientes. Maquetala para que parezca una lista de tareas de una aplicación (pej, alternando para cada elemento el color de fondo y el color del texto blanco y verde).

Añade varios botones debajo de la lista:

- **Añadir Tarea al Inicio:** pide el nombre de la tarea usando prompt y la añade al inicio de la lista (debe quedar maquetada).
- **Añadir Tarea al Final:** igual que el botón anterior pero añade la nueva tarea al final de la lista (debe quedar maquetada).
- **Eliminar Ultima Tarea:** Cuando el usuario pulse el botón, debe eliminarse el último elemento de la lista.
- **Eliminar Primera Tarea:** Igual que el anterior pero ahora se elimina la primera tarea de la lista.

IMPORTANTE: Si ya no hay elementos, debe mostrarse un mensaje por consola indicando que la lista está vacía.

25.- Crea el efecto ‘ampliación’ de algunas galerías de fotos que podemos encontrar por internet.

Para empezar, busca una imagen de gran tamaño (unos 1280x720px al menos) y coloca la imagen en HTML pero con la mitad de tamaño. Crea el código Javascript necesario para que , al hacer click sobre la imagen, suceda lo siguiente:

- Aparecerá una capa semitransparente de color gris que tapaná TODA la parte visible de la página y que no se moverá de sitio al hacer scroll.
- Dentro de esa capa, aparecerá la imagen con su tamaño original centrada tanto horizontal como vertical y perfectamente visible.
- Al hacer click en cualquier punto de la capa, desaparecerá junto con la imagen de tamaño original.

Nota: Haz uso del objeto Screen y/o Window.

b) Mejora el ejercicio anterior de modo que funcione para cualquier imagen que haya en una página. Al hacer click sobre cualquiera de las imágenes de la página, esta aparece con su tamaño original dentro de la capa semitransparente que lo cubre todo.

Formularios

1.- Crea un formulario con dos campos de texto y un botón llamado *copiar*. El segundo campo de texto debe estar colocado en modo de solo lectura.

Desarrolla un programa que, cuando se pulse el botón *copiar*, copie todo el contenido del primer campo de texto en el segundo.

2.- Crea un formulario con dos campos de texto. El segundo campo de texto debe estar colocado en modo de solo lectura.

Desarrolla un programa que, conforme se vaya escribiendo en el primer campo, se va a ir copiando ese contenido en el segundo campo. *Nota: busca el evento adecuado para este ejercicio.*

3.- Crea un formulario con varios checkbox (pej, aficiones), un textarea y un botón. Implementa un programa que, tras pulsar el botón se muestre dentro del textarea todas las opciones marcadas.

b) Mejora el apartado anterior quitando el botón. Ahora cada vez que se marque un checkbox, se añadirá al contenido del textarea. Si se desmarca, hay que quitarlo.

4.- Realiza un programa que, haciendo uso de dos campos de texto y un botón, realice la conversión de dólares y euros ($1\$ = 0.83\text{€}$).

Completa el ejercicio añadiendo dos campos y un botón nuevos para implementar la conversión contraria (de euros a dólares).

b) Mejora el ejercicio anterior usando solo dos campos y un botón para realizar ambas conversiones. Para saber qué conversión debemos realizar en cada momento, esta se debe seleccionar de una lista desplegable.

5.- Crea un formulario con dos campos de texto y un botón llamado COMPROBAR. El botón debe comprobar si el contenido de uno de los campos es anagrama del otro. Mostrará el resultado haciendo uso de la función alert. Si ambos campos están vacíos, no se considerará que son anagramas.

Nota: una palabra es anagrama de otra si las dos tienen las mismas letras, con el mismo número de apariciones, pero en un orden diferente.

6.- Crea un formulario con varios elementos textbox (pej, países) y una lista desplegable vacía.

Implementa el código necesario para que, cada vez que se marque un checkbox, se cree una entrada nueva en la lista desplegable con el país correspondiente. Si se desmarca un checkbox, debe eliminarse el país correspondiente de la lista.

7.- Crea un textarea y un botón llamado *Publicar*. Cada vez que se pulse el botón *Publicar*, se va a crear un div cuyo contenido será el texto del textarea. *Nota: maqueta de manera vistosa los DIV creados.*

- b) Haz que sólo se publiquen en el div los 50 primeros caracteres escritos en el textarea.
- c) Limita a que sólo se puedan escribir 100 caracteres en el textarea. Después de los 100 caracteres escritos, no se puede seguir escribiendo. El botón Publicar no debe cambiar su comportamiento. *Nota: busca el evento adecuado para este ejercicio.*

8.- Coloca un div que ocupe toda la pantalla, que tenga una altura de 200px y un color de fondo (el que quieras). Crea un programa que permita cambiar el color de ese DIV por otros usando:

- a) Indicando el color deseado entre alguna de las opciones de un bloque de botones radio (*al menos 3 colores para elegir*).
- b) Indicando el color deseado entre los elementos de una lista desplegable (*al menos 3 colores para elegir*).

9.- Crea un campo de texto en el que se deba colocar un DNI (nueve dígitos y una letra). Cuando ese elemento pierda el foco (crea más elementos para que eso ocurra), se debe validar lo siguiente:

- Que hay nueve números y una letra.
- Que la letra introducida es la letra correcta (*Aquí se indica: <https://www.interior.gob.es/opencms/ca/servicios-al-ciudadano/tramites-y-gestiones/dni/calculo-del-digito-de-control-del-nif-nie/>*)

10.- Crea un formulario con dos listas desplegables y un botón. La primera lista desplegable tendrá los valores: vacío (el valor por defecto), Intel, AMD y Nvidia.

La segunda lista inicialmente no tendrá ningún elemento. Al elegir un valor en la segunda, la segunda lista se llenará con los valores siguientes:

- Para Intel: HD 3000, HD 4000, IRIS 600.
- Para AMD: RX Series 500, Vega Series, RX Series 6000.
- Para Nvidia: GTX Serie 1000, GTX Serie 2000, GTX Serie 3000.

Además, al pulsar el botón, este debe comprobar que ambas listas tienen un valor seleccionado e informará usando un alert de si eso ocurre o no.

11.- Crea un formulario con los siguientes campos:

- DNI: 8 números y una letra.
- Nombre: uno o dos nombres.
- Apellidos: uno o dos apellidos.
- Fecha de nacimiento: formato dd/mm/yyyy.
- Web personal: formato `https://www.algo.tresletras`
- Contraseña: entre 8 y 12 caracteres.
- Un botón para resetear el formulario.
- Un botón para enviar el formulario.

Haciendo uso de JavaScript pero sin usar expresiones regulares¹, cuando se pulse el botón correspondiente, valida el formulario siguiendo las normas que se detallan a continuación y envía el resultado a otra página que muestre un mensaje de que todo ha salido bien.

El formulario estará correcto sí y solo sí:

- Ningún campo se queda vacío
- El DNI tiene una longitud de 9
- La fecha de nacimiento tiene el formato correcto.
- Hay una o dos palabras en los campos nombre y apellidos.
- El campo de la web debe empezar por `https://`
- La longitud de la contraseña es correcta.

Si algún campo se rellena de forma errónea, debe indicarse ese error usando un alert y devolviendo el foco al elemento erróneo (o al primer elemento erróneo).

Como se ha comentado antes, debes preparar una sencilla página html con un mensaje de que todo ha ido bien y enlazarla al formulario de tal modo que aparezca cuando todos los campos sean correctos.

¹ como no puedes hacer uso de expresiones regulares, usa funciones para cadenas.

Control de Eventos

1.- Crea un documento HTML con un `<section>` que en su interior tiene un párrafo y este a su vez, tiene un enlace al campus del instituto. (Maqueta los elementos para que se distingan unos de otros).

Todos los elementos mencionados tienen una función asociada al evento click del ratón. El section cambiará su color de fondo, el párrafo cambiará su anchura y el enlace mostrará un panel `confirm` para decirle al usuario si se mueve a la página enlazada o no (es decir, al pulsar el enlace este no va a la página enlazada a no ser que lo diga el usuario).

- a) Prueba la fase de burbuja y la fase de captura ¿ves la diferencia?
- b) Con todas las funciones de evento definidas y asignadas: haz que solo se ejecute la del enlace.
- c) Con todas las funciones de evento definidas y asignadas: haz que solo se ejecute la del section.

2.- Crea un documento HTML con dos DIV. Crea un script que, cada vez que se mueva el ratón, indique las posiciones de este de la forma (posx,posy). Usa el primer div para mostrar esa información.

Además, cuando se pulse alguno de los botones del ratón, en el segundo DIV se debe mostrar qué botón se ha pulsado (Izquierdo , derecho o central).

También debe impedir que aparezca el menú contextual al pulsar el botón derecho del ratón.

3.- Crea un documento HTML que tenga de un título h1 con el texto `<<Pulsa una tecla>>`, y un DIV central de 300x300px maquetado para que se vea bien (borde, tamaño de letra alto, color de fondo...) Utiliza el evento `keypress` para determinar qué carácter ha escrito el usuario utilizando el teclado y muéstralo dentro del div.

4. Crea un documento HTML que conste de un título h1 con el texto `<<Usa los cursores>>`, y un DIV de tamaño 100x100px (maquetado con borde color negro y color de fondo rojo). El DIV debe aparecer en una posición aleatoria del documento cada vez que se recarga la página.

Utiliza el evento `keypress` para determinar si el usuario pulsa una tecla del cursor, y en caso afirmativo, utilizando CSS haz que el DIV se desplace 20px en la dirección de flecha elegida por el usuario. Evita que se salga de los bordes del documento.

b) Ahora, en vez de hacer que el DIV se mueva 20px en una de las direcciones al pulsar la tecla correspondiente, el DIV se debe mover en la dirección indicada de 20 en 20 px y no parar hasta que encuentre un borde o hasta que se cambie el sentido de la dirección al pulsar otra tecla de los cursores.

Expresiones Regulares

1.-¿Qué validan las siguientes expresiones regulares?

- a) `ho*a`
- b) `([B-D] | [F-H] | [J-N] | [P-T] | [V-Z])`
- c) `^[0-9]{5}$`
- d) `[0-4][0-9][0-2][0-9]{3}`
- e) `([0-4][0-9]|5[0-2])[0-9]{3}`
- f) `(^[a-z])(.*[A-Z])(.*\d)[a-zA-Z]{8,12}$`

2.- Sea la expresión regular: `/^[A-J]\w+\ses\s\w+/` Indica qué cadenas (o parte de ellas) cumplen el patrón indicado.

- a) Jaime no es guapo.
- b) Jose es un llorón.
- c) Guille deja de hablar.
- d) Paco es muy divertido.
- e) Enya es pizpireta.
- f) Sebas es rubio.
- g) Isma es callado.

3.- ¿Qué validan las siguientes expresiones regulares?

- a) `\d{6}$`
- b) `^[a-z0-9-]+$`
- c) `^95[0-9]\s*(\d{6}$)`
- d) `(\d{2}\s){2}`
- e) `(\d{2}\s){2}\d{2}$`
- f) `^95[0-9]\s*(\d{2}\s){2}\d{2}$`
- g) `(\d{3}\s){2}`
- h) `^95[0-9]\s*(\d{3}\s){2}`
- i) `95[0-9]\s* ((\d{6}$) | ((\d{2}\s){2}\d{2}$))`
- j) `95[0-9]\s* ((\d{6}$) | ((\d{2}\s){2}\d{2}$) | ((\d{3}\s){2}))`

4.- Crea un script que valide un DNI español usando expresiones regulares. El DNI está compuesto de 8 dígitos entre el cero y el nueve y una letra mayúscula.

5.- Indica cual sería la expresión regular necesaria para validar un email. Recuerda:

- El email debe de empezar por una cadena de caracteres que no empiecen por @ o espacio en blanco. La longitud de esta cadena debe ser mayor a cero.
- A continuación, debe aparecer el símbolo @.
- Después, una cadena de caracteres que no empiecen por @ (arroba), "." o espacio en blanco, cuya longitud sea mayor a cero.
- Esa cadena debe estar seguida de un "." (punto)
- Después, otra vez, una cadena de caracteres que no empiecen por @, "." o espacio en blanco, cuya longitud sea mayor a cero.

6.- Indica cuáles serían las expresiones regulares que hay que usar para validar las siguientes entradas de un formulario:

- a) Nombre de usuario: debe empezar por letra (mayúscula o minúscula). Puede tener números y los caracteres guion medio y guion bajo (ningún otro). No puede contener espacios. Longitud máxima de 16 caracteres.
- b) Contraseña: longitud mínima de 8 caracteres y máxima de 18. Puede contener números, espacios y cualquier carácter. Debe contener alguna mayúscula.
- c) Valor hexadecimal: debe comenzar por el carácter almohadilla # o 3 caracteres hexadecimales válidos: letras de la A a la F (mayúsculas o minúsculas) y números del 0 al 9. Nada de espacios o algún otro carácter.
- d) URL: comienza por `http://` o `https://`. A continuación puede aparecer o no la cadena `www`. Después irá el nombre del dominio (números, letras y guion medio). Punto. Puede aparecer o no el subdominio (lo mismo que el dominio). Punto. Y finaliza con la extensión (2 o tres letras. No números ni caracteres). La extensión puede repetirse.
- e) Una IP válida: cuatro bloques de dígitos separados por el carácter punto (el último bloque no lleva el punto). En cada bloque puede aparecer 1, 2 o 3 dígitos. Ninguna letra o carácter están permitidos en los bloques.
 - Mejora el apartado anterior para que en los bloques solo puedan aparecer números entre el 0 y el 255.
- a) Etiqueta HTML: comienza por el signo `<`. A continuación lleva letras mayúsculas o minúsculas (nada de números o caracteres). Finaliza con el carácter `/` (puede aparecer o no) y con el carácter `>`.
 - Mejora el apartado anterior para que pueda reconocer atributos: `atributo="valor"`. Recuerda que se pueden poner varios atributos separados por espacio.

Objetos

Parte I

1. Crea un objeto TUTOR, el cual tendrá los campos: nombre, edad, DNI y título universitario.
2. Crea un objeto ASIGNATURA. De cada asignatura necesitamos saber su nombre, el curso al que pertenece (un número entero) y el número de horas totales. A continuación crea una lista de, al menos, 4 objetos asignaturas diferentes.
3. Haciendo uso del objeto **tutor** creado en el ejercicio 1 y de la lista de objetos **asignatura** creada en el ejercicio 2, define un objeto ALUMNO.
Un **alumno** tendrá un nombre, una edad, el ciclo que estudia, el curso en el que está (un número entero), un tutor, una lista de **asignaturas** y una lista de notas media de cada asignatura.
La lista de notas debe ser un array de números, y cada nota se corresponderá con la asignatura del mismo índice en la lista de asignaturas.
4. Haciendo uso de las distintas estructuras FOR recorre el objeto **alumno** y muestra, dentro de un DIV maquettato en CSS, todos los datos del alumno.
Imagina que estás haciendo una aplicación para gestionar alumnos que va a usar gente que no sabe programar. Es decir, muestra la información del alumno de la forma más clara posible.
No se puede escribir HTML directamente desde JavaScript usando `innerHTML` con bloques grandes. Se deben crear los elementos con `createElement`.

Parte II

5. Añade las siguientes funciones al objeto TUTOR:
 - `mostrar()` : devuelve una cadena con todos los datos del objeto (intenta que la información devuelta sea clara).
 - `cambiarNombre(nuevo)` : permite cambiar el nombre del TUTOR.
6. Añade las siguientes funciones al objeto ASIGNATURA:
 - `mostrar()` : devuelve una cadena con todos los datos del objeto (intenta que la información devuelta sea clara).
 - `cambiarHoras(nueva)` : permite cambiar el número de horas del objeto.

7. Añade las siguientes funciones al objeto ALUMNO:

- `calcularMedia()` : en base a la lista de medias del alumno, devuelve la media total del curso de ese alumno.
- `MediaAsignatura()` : devuelve una cadena con el nombre de cada asignatura y, a continuación, su media.
- `mostrar()` : devuelve una cadena con todos los datos del objeto (intenta que la información devuelta sea clara). Incluye las asignaturas con sus medias y la media total calculada en los puntos anteriores.

Nota: sería interesante que practicaras el crear métodos para mostrar/cambiar campos del objeto ALUMNO.

8. Haciendo uso de las funciones implementadas en los ejercicios anteriores (y de algún FOR) recorre el objeto ALUMNO y muestra, dentro de un DIV maquettato en CSS, todos los datos del alumno.

Imagina que estás haciendo una aplicación para gestionar alumnos que va a usar gente que no sabe programar. Es decir, muestra la información del alumno de la forma más clara posible.

WebStorage

Ejercicio 1: Crea una página web sencilla con un formulario que contenga dos campos: "Nombre" y "Clave".

A continuación, escribe un código JavaScript que almacene los datos del formulario en el `localStorage` del navegador cada vez que se envía el formulario.

Crea OTRA página que, al cargarse, muestre en su cuerpo los datos del `localStorage`.

Añade en esa página el botón BORRAR que borre los datos almacenados en el `localStorage` y un enlace para volver a la página inicial.

Ejercicio 2: Crear un script que sirva como selector de tema de una página: tema claro (por defecto) y tema oscuro.

El tema se debe elegir usando un elemento `select` junto con un botón llamado "Activar Tema".

El botón debe almacenarse usando los objetos `webStorage` del navegador. Además, la opción elegida por el usuario debe permanecer aunque se recargue o se cierre la página.

Por tanto, lo primero que debe hacer el script es mirar si hay un tema almacenado y, en caso afirmativo, aplicarlo.

Ejercicio 3: Crear una página web sencilla con un formulario compuesto de un campo de texto llamado 'Nueva Tarea', un botón llamado 'Añadir' y otro botón llamado 'Borrar'. Crea también un DIV debajo del formulario que tenga como título: *Lista de Tareas Pendientes*.

El botón 'Añadir' debe encargarse de enviar el formulario a la misma página donde está creado (es decir, en el `action` del formulario, pon el mismo archivo que estás trabajando)

A continuación, crea un script que haga lo siguiente:

- Lo primero que debe hacer el script es mirar si hay algo almacenado en `localStorage`.
 - Si no hay nada, muestra el HTML sin hacer nada más
 - Si hay algo, muestra todo el contenido de `localStorage` en el DIV
- Cada vez que se pulse el botón 'Añadir', si el campo de texto no está vacío, se añadirá su contenido al (como si se registrase una tarea nueva en el sistema).
- El Botón 'Borrar', elimina todas las tareas almacenadas en el `localStorage` tras pedir confirmación al usuario.

Ejercicio 4: Crea una pagina web con un formulario que pida el Nombre, los Apellidos, la edad y el Email al usuario.

Cada vez que se envía el formulario, esos datos deben almacenarse en un objeto JSON y almacenarlos en el almacenamiento local del navegador.

Crea otra página con dos botones:

1. Recuperar: al pulsarlo, recupera el objeto almacenado y muestre cada VALOR de las propiedades del objeto en diferentes etiquetas section. *Nota: si se pulsa varias veces, irá creando nuevos Section.*
2. Borrar: borra el contenido de la localStorage.

Ejercicio 5: Carrito de la compra

Para este ejercicio debes usar los archivos HTML y CSS dentro del archivo **Ejercicio05-webStorage.zip** que se encuentra en la plataforma del curso.

Crea un Script que haga lo siguiente:

- Crea un objeto literal vacío.
- Cada vez que el usuario seleccione un elemento en la lista desplegable y pulse el botón “Añadir al carrito”:
 - Añade ese elemento al objeto literal
 - Guarda (o sustituye) ese objeto literal en la `webStorage`
 - Lee la `webStorage` y muestra su contenido en la zona llamada “*Carrito de la Compra*”
- Cada vez que el usuario pulse el botón “Pagar”:
 - Si el carrito de la compra está vacío: se muestra un alert indicando que no hay nada en el carrito.
 - Si no está vacío el carrito: el script se moverá a otra página `pasarela.html` y volcará la lista de elementos comprados en el div correspondiente (olvida el resto de la web)

Importante: NO USES FORMULARIOS en este ejercicio para moverte de una pagina a otra. Usa el objeto `location` del navegador (concretamente `location.href`)

Ejercicio 6: Crea una página web con un formulario de inicio de sesión que contenga los campos "Usuario" y "Contraseña".

Cuando se pulse el botón enviar:

- Valida el nombre de usuario sea válido mediante el uso de expresiones regulares. El nombre de usuario debe comenzar con una letra minúscula, seguida de letras minúsculas y/o números, y tener una longitud mínima de 8 caracteres.
- Valida la contraseña sea válida mediante el uso de expresiones regulares. La contraseña debe tener al menos 8 caracteres, incluyendo al menos una letra mayúscula, una letra minúscula y un número.
- Si alguno de los campos falla, ponle un borde rojo de tamaño 2px.
- Almacena el nombre de usuario y la contraseña en FORMA DE OBJETO en el almacenamiento local del navegador.

En la misma página del formulario, escribe código JavaScript para recuperar el objeto con el nombre de usuario y la contraseña desde el almacenamiento local cada vez que se carga la página y rellena automáticamente los campos del formulario con los datos almacenados.

Si no existieran datos en el almacenamiento local, los campos del formulario quedaran vacíos.

Añade el botón BORRAR para eliminar el contenido de la localStorage

Personajes de Dragon Ball

Goku  Raza: Saiyan Máximo Ki: 90 Septillion	Vegeta  Raza: Saiyan Máximo Ki: 19.84 Septillion	Piccolo  Raza: Namekian Máximo Ki: 500.000.000	Bulma  Raza: Human Máximo Ki: 0	Freezer  Raza: Frieza Race Máximo Ki: 52.71 Septillion	Zarbon  Raza: Frieza Race Máximo Ki: 30.000	Dodoria  Raza: Frieza Race Máximo Ki: 20.000
		Ginyu 	Celula 	Gohan 		

Asincronía

Goku

El protagonista de la serie, conocido por su gran poder y personalidad amigable. Originalmente enviado a la Tierra como un infante volador con la misión de conquistarla. Sin embargo, el caer por un barranco le proporcionó un brutal golpe que si bien casi lo mata, este alteró su memoria y anuló todos los instintos violentos de su especie, lo que lo hizo crecer con un corazón puro y bondadoso, pero conservando todos los poderes de su raza. No obstante, en la nueva continuidad de Dragon Ball se establece que él fue enviado por sus padres a la Tierra con el objetivo de sobrevivir a toda costa a la destrucción de su planeta por parte de Freeza. Más tarde, Kakarot, ahora conocido como Son Goku, se convertiría en el príncipe consorte del monte Fry-pan y líder de los Guerreros Z, así como el mayor defensor de la Tierra y del Universo 7, logrando mantenerlos a salvo de la destrucción en innumerables ocasiones, a pesar de no considerarse a sí mismo como un héroe o salvador.



● Planeta de origen

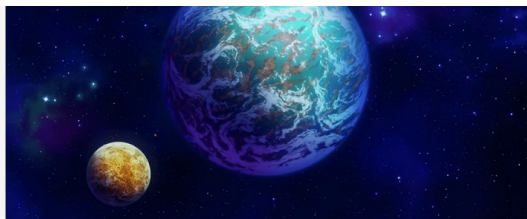
Vegeta

El planeta Vegeta, conocido como planeta Plant antes del fin de la Guerra Saiyan-tsufruiiana en el año 730, es un planeta rocoso ficticio de la serie de manga y anime Dragon Ball y localizado en la Vía Láctea de las Galaxias del Norte del Universo 7 hasta su destrucción a manos de Freezer en los años 737-739. Planeta natal de los Saiyans, destruido por Freezer. Anteriormente conocido como Planeta Plant.

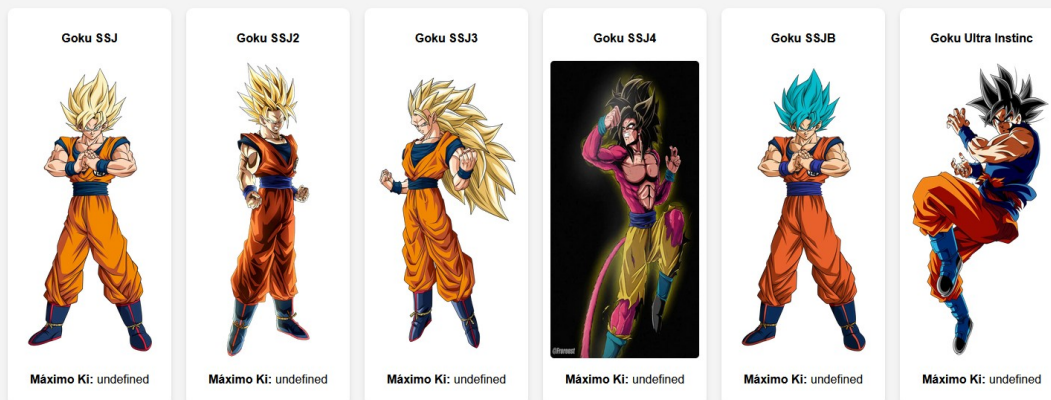


1.- Crea una página web sencilla que tenga un botón y un <section> con id de nombre “salida” que se usará para colocar los resultados.

El planeta Vegeta, conocido como planeta Plant antes del fin de la Guerra Saiyan-tsufruiana en el año 730, es un planeta rocoso ficticio de la serie de manga y anime Dragon Ball y localizado en la Vía Láctea de las Galaxias del Norte del Universo 7 hasta su destrucción a manos de Freezer en los años 737-739. Planeta natal de los Saiyans, destruido por Freezer. Anteriormente conocido como Planeta Plant.



Transformaciones



[— Volver a personajes](#)

Haz que cuando se pulse el botón se realice una consulta a la URL <https://jsonplaceholder.typicode.com/comments> y muestre dentro del elemento “salida” cada uno de los comentarios obtenidos (campo body)

2.- Haciendo uso de la API de Dog CEO (<https://dog.ceo/dog-api/>) crea una galería de perros. Para ello:

- Implementa un archivo GaleriaDOG.html que posea, al menos, un contenedor donde irá ubicada la imagen y un botón para obtener una nueva imagen.
- Por defecto, al cargar la pagina, la galería debe tener una imagen cargada de la API.
- Cada vez que el usuario pulse el botón, la imagen debe cambiar.

3.- Crea una página web sencilla que tenga dos elementos <section> y un botón. Nota: no coloques id a ninguno de los <section>

Cuando se pulse el botón, este queda deshabilitado y debe realizar una solicitud a la API de FakeStore (<https://fakestoreapi.com/products>). Con lo que devuelva, se debe listar en el primer <section> de la pagina, el nombre y el precio de todos los productos obtenidos.

Hay que implementar también que, al pasar el ratón por encima de alguno de los productos listados, en el segundo <section> de la página debe aparecer la imagen de dicho producto.

4.- Arregla el ejercicio de la web de la librería para que ahora consuma el objeto desde la siguiente URL: <http://www.jaimeweb.es/medac/books.json>

5.- Crea una aplicación que controle una lista de tareas consumida desde la URL <https://jsonplaceholder.typicode.com/todos> La aplicación debe listar las tareas, agregar nuevas tareas (solo en local) y ser capaz de cambiar el estado de la tarea indicada.

Para que la estructura de la aplicación sea lo más homogénea posible, se deben respetar las siguientes pautas:

- Los campos necesarios de cada tarea serán `title` y `completed`, aunque el alumno podrá hacer uso de la propiedad `id` si así lo cree conveniente.
- Todas las tareas serán mostradas dentro de una lista desordenada con `id = "tasks"`
- Antes de la lista de tareas debe aparecer un formulario destinado a la creación de una tarea nueva. Ese formulario solo tendrá dos campos: un campo de texto para explicar la tarea y un botón para añadir dicha tarea. *Nota: la tarea debe guardarse en la estructura almacenada en memoria y luego aparecer en la lista de tareas.*
- Las tareas podrán estar en dos estados: "sin terminar" y "completada". Al hacer click sobre una tarea en la lista de tareas, esta debe cambiar de estado.

La personalización de la aplicación mediante CSS se deja a elección del alumno.

6.- En este ejercicio vamos a hacer uso de la API de Dragon ball (<https://dragonball-api.com/>) para crear una web que muestre la información de los personajes de la serie.

- Al iniciar, la página debe cargar los personajes que devuelve el endPoint <https://dragonball-api.com/api/characters> y solo debe poseer en el body un `<section>`
- Esos personajes serán mostrados dentro de un `article` el cual será hijo del único `section` de la página.
- En cada `article` se mostrará el nombre del personaje, su imagen, la raza y el maximoKi que posee.
- **PARTE COMPLICADA:**
- Al hacer click sobre un `article`, se cargará una página destinada a dar todos los datos sobre el personaje elegido.

- En esa página, se mostraré el nombre del personaje en grande y justo debajo su descripción y la imagen del personaje (ambas cosas en la misma línea).
- Más abajo debe aparecer el lugar al que pertenece: Nombre del planeta, descripción e imagen.
- Finalmente, se mostrarán sus transformaciones si las tuviera a modo de fichas con los datos: Nombre de la transformación, imagen y maxKi.
- Al final de esa página debe aparecer un enlace o botón para volver a la pagina inicial donde se muestra la lista de personajes.

Nota: mira la documentación de la API porque vas a necesitar, al menos, los endPoints: <https://dragonball-api.com/api/planets> y <https://dragonball-api.com/api/transformation>

A continuación se muestran pantallas de ejemplos de diseño:

